

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

1. Project Overview	Error! Bookmark not defined.	2. Background & Business Objective
33.	Product	Purpose
44.	System	Functionalities
45.	User Stories & Use Cases	
66.	Features Included (Scope Definition)	
77.	Features Not Included (Scope Control)	
88.	Assumptions & Constraints	
89.	Risks & Questions Throughout Development	

Project Name: Risk-Aversion-Steward

Version: 2.0

1. Project Overview

- **Problem Statement:** Small-to-Medium Businesses (SMBs) and warehouse managers constantly struggle with reactive inventory management, leading to severe capital lockup (dead stock) or lost revenue (stockouts). Furthermore, manual data entry for tracking outbound sales via messy supplier invoices is highly tedious, error-prone, and slow. Liquidating aging stock is often based on guesswork rather than data-driven strategy.
- **Target Domain:** Supply Chain Risk Management, Automated Retail Operations, and AI-driven Resource Allocation.
- **Proposed Solution Summary:** Inventory Steward removes the operational friction by acting as an AI co-pilot for warehouse management. It predicts 15-day stockout risks, auto-generates supplier purchase orders via email, uses an "AI Sales Scanner" to extract outbound data from uploaded invoices (PDF/Word/Excel) instantly, and provides AI-generated liquidation strategies to recover cash from dead stock. It shifts the user's role from a manual data-entry clerk to a strategic decision-maker.

2. Background & Business Objective

- **Background of the Problem:** For many years, inventory management followed a manual, linear pathway. A manager checks the shelves, manually types data from PDF/Excel invoices into an ERP system, and manually drafts emails to suppliers to restock. When stock doesn't sell, they blindly apply a 50% discount. This process requires extensive labor work, is prone to human error, and lacks proactive risk mitigation.
- **Importance of Solving This Issue:** Accelerating the time to process inventory from hours to seconds protects business cash flow. By breaking the technical barrier with AI invoice

scanning, anyone can update stock levels without training. By predicting stockouts, businesses never miss a sale. By intelligently clearing dead stock, businesses can recover thousands of dollars in frozen capital.

- **Strategic Fit / Impact:** It aligns with the AI-Native Business Ecosystem. By integrating large language models (ILMU-GLM / Z.ai GLM) to handle unstructured data (like invoices) and generate complex business strategies, the system transforms raw data into actionable, automated workflows.

3. Product Purpose

3.1. Main Goal of the System: To provide an AI-enabled inventory manager that autonomously mitigates supply chain risks and eliminates manual data entry.

3.2. Intended Users (Target Audience):

- Warehouse Managers
- Retail Store Owners / SME Founders
- Supply Chain Analysts
- Financial Controllers

4. System Functionalities

4.1. Description: The system operates as a high-speed risk diagnostic and generative engine for inventory control. Using a FastAPI backend and a responsive dashboard, it captures relationships between stock levels, sales velocity, and unstructured invoice documents to execute automated operational actions.

4.2. Key Functionalities:

- **AI Sales Scanner:** Translates unstructured human-readable invoices (PDF, DOCX, XLSX) into structured JSON data to automatically deduct stock levels in bulk.

- **Smart Sourcing & Auto-Order:** Monitors the 15-day stockout forecast and enables 1-click automated Restock Email generation sent directly to suppliers via SMTP. Features a fail-safe cancellation rollback mechanism
- **AI Clearance Strategy:** Evaluates aging/stagnant stock and uses LLM to recommend actionable, step-by-step liquidation plans (e.g., Flash Bundles, BOGO) alongside success probabilities and expected capital recovery metrics.
- **Live Risk Scan:** Performs a global diagnostic of the database to instantly categorize products into "Urgent" or "Healthy," automatically prompting the user to take action.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: The system utilizes **ILMU-GLM-5.1** as the primary model, with **Z.ai GLM-4-Flash** as the fallback. These models were chosen for their exceptional ability to parse messy, unstructured text (like OCR'd PDFs) and accurately format the output into strict JSON arrays, which is a core requirement for our automated database updates.

4.3.2. Prompting Strategy: We utilize a Few-Shot & Context-Injected Prompting strategy. By injecting the actual "System SKUs" list directly into the prompt context, we ground the AI. This ensures the AI matches the random product descriptions found on customer invoices strictly to our internal database SKUs, preventing hallucination.

4.3.3. Context & Input Handling: The system handles oversized invoice inputs by extracting text (via PyMuPDF, Pandas, python-docx), aggressively cleaning non-ASCII characters to prevent model freezing, and truncating the text to the first 800-1000 characters. Since invoice tables usually appear on the first page, this truncation prevents token overflow and drastically reduces API response times.

4.3.4. **Fallback & Failure Behavior:** The system features a highly robust 3-tier failure mechanism:

1. Tries ILMU-GLM (15s timeout).
2. If it fails/times out, seamlessly falls back to Z.ai GLM.
3. If all APIs are unreachable, it triggers a "Smart Demo Fallback" (Rule-based engine), which generates localized inventory data/strategies to ensure the user experience and business operations are never interrupted by server outages.

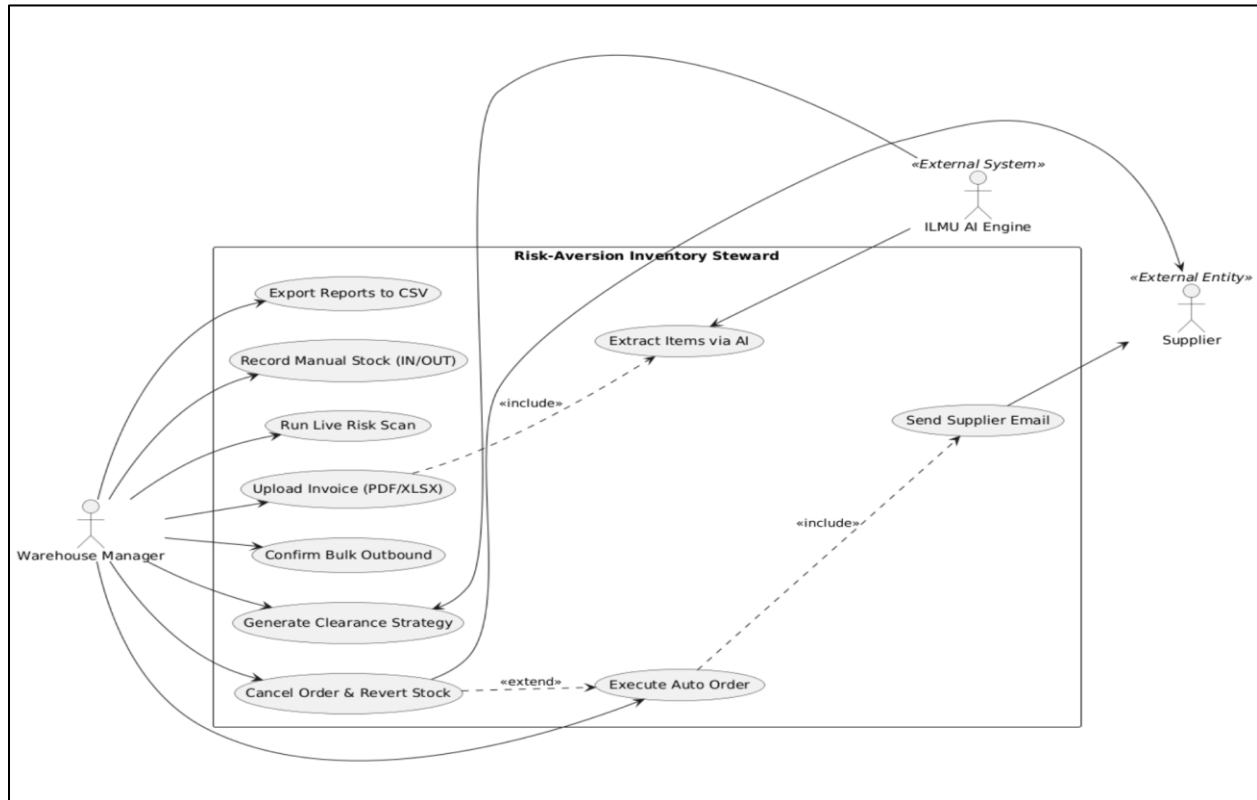
5. User Stories & Use Cases

- **User Stories:**
- *Example 1: "As a warehouse manager, I want to drag and drop a PDF invoice into the system so that it automatically extracts the SKUs and quantities sold without me typing them."*
- *Example 2: "As a business owner, I want the system to alert me when an item has less than 15 days of stock and automatically draft a purchase order email to my supplier."*
- **Use Cases (Main Interactions):**

Invoice to Outbound Flow: User uploads a PDF. The Backend parses text -> LLM identifies SKUs and Qty -> System presents a Review Table -> User clicks "Confirm Bulk Outbound" -> Database updates and creates a transaction log.

Automated Procurement Flow: User opens "Sourcing" tab -> Clicks "Auto Order" on a low-stock item -> Passes Double-Confirmation warning -> Backend executes UPDATE inventory and triggers SMTP email to the supplier.

- **Use Case Diagram:**



The Use Case Diagram illustrates the interactions between the Warehouse Manager, the ILMU AI Engine, and external Suppliers. The Manager handles core tasks such as initiating Auto Orders and uploading invoices. The system utilizes <<include>> relationships to demonstrate how the AI Engine is mandatorily invoked during invoice processing and clearance generation. Additionally, an <<extend>> relationship is used for the 'Cancel Order' function, showcasing the system's fail-safe mechanism that allows users to optionally revert accidental purchase orders and notify suppliers automatically

6. Features Included (Scope Definition)

- Dashboard with 15-Day Stockout Forecasting and real-time tracking.
- AI Sales Scanner supporting unstructured .pdf, .docx, and .xlsx inputs.

- 1-Click Automated Supplier Emailing (with Order Cancellation & Reversal feature).
- AI Clearance Strategy generator for dead stock.
- Transaction Reports logging with direct Export to CSV (BOM formatting for Excel compatibility).

7. Features Not Included (Scope Control)

- Implementation of full ERP integration or payment gateway processing.
- Pure Image OCR (Scanning photos of receipts requires an external vision model because current scope handles digital text-based documents).
- Multi-user Role-Based Access Control (RBAC), the current system is built for a single administrative environment.

8. Assumptions & Constraints

- **LLM Cost Constraint:** Processing entire 20-page PDFs is too slow and costly. We assume invoice items are at the beginning of documents, heavily truncating text to keep inference costs low and response times under 5 seconds.
- **Technical Constraints:** Automated emails rely on standard SMTP configurations. If the host blocks port 587, emails will fail (the system will log a simulation instead to prevent crashing).
- **User Input:** The model is not allowed to blindly execute stock deductions; all AI-extracted invoice items require a human-in-the-loop "Confirm Bulk Outbound" click before touching the database.

9. Risks & Questions Throughout Development

- **Hallucination Risk:** How are we ensuring that AI doesn't invent SKUs that don't exist in our database?
- : Mitigated by forcing the AI to select only from a dynamically injected JSON array of valid System SKUs in the prompt.
- **Format Stability:** What happens if the invoice contains hidden control characters or corrupted text?

: Mitigated by a Regex sanitization step `re.sub(r'^\x00-\x7F]+', ' ', text)` in the backend before sending the prompt to the AI.

- **System Reliability:** What if the Hackathon internet is slow and API times out?

: Mitigated by the 3-tier fallback architecture, guaranteeing a successful demo flow regardless of external API health.